



Verm (from vermilingua)

Diseño e Implementación de un Lenguaje
(Difficulty: Bro-Force)

1. Equipo

Nombre	Apellido	Legajo	Email
Uriel	Sosa Vázquez	63250	usosavazquez@itba.edu.ar
Felipe	Venturino	63161	fventurino@itba.edu.ar

2. Repositorio

<https://github.com/urielsv/vermilingua>

3. Dominio

Desarrollar un lenguaje que permita analizar y filtrar tráfico de red desde archivos de captura (PCAP). El lenguaje permitirá definir reglas para extraer información específica de diversos protocolos (TCP/IP, HTTP, etc.), generar estadísticas sobre el tráfico, y detectar patrones o anomalías en los paquetes.

Para reducir la complejidad del proyecto, el análisis se centrará en los protocolos más comunes (IPv4, TCP, UDP, HTTP) y se utilizará una librería (libpcap) para el procesamiento de los archivos PCAP. El compilador transformará el programa de entrada en un ejecutable que procese los PCAP y genere reportes estructurados (JSON) que puedan ser consumidos por otras aplicaciones o visualizados mediante una interfaz HTML simple.

Esta implementación permitiría a profesionales de redes y seguridad detectar problemas de comunicación, analizar rendimiento e identificar comportamientos anómalos sin necesidad de dominar herramientas complejas como Wireshark.

4. Construcciones

- (I). Se podrán definir fuentes de captura desde un archivo PCAP.
- (II). Se podrán crear filtros BPF (Berkeley Packet Filter) para limitar qué paquetes serán procesados.
- (III). Se podrán extraer campos específicos de los protocolos soportados (IP, TCP, UDP, HTTP).
- (IV). Se podrán definir variables para almacenar información extraída de los paquetes.
- (V). Las variables podrán ser de tipo packet, address, integer, boolean, string o timestamp.
- (VI). Las variables podrán ser vectores.
- (VII). Se proveerán operadores relacionales como $<$, $>$, $=$, $\neq$, $\leq$ y $\geq$.
- (VIII). Se proveerán operaciones aritméticas básicas como $+$, $-$, $*$ y $/$.
- (IX). Se proveerán operaciones lógicas básicas como AND, OR y NOT.
- (X). Se proveerán estructuras de control básicas de tipo IF-ELSE, FOR y WHILE.
- (XI). Se podrán definir funciones de agregación (count, sum, avg, min, max) sobre conjuntos de paquetes.
- (XII). Se podrán agrupar resultados por campos específicos (ej: por IP origen, por puerto destino).
- (XIII). Se podrán definir reportes basados en los datos extraídos.
- (XIV). Se podrá exportar un reporte a un archivo JSON para su posterior procesamiento o visualización con HTML.

5. Casos de prueba

Casos de aceptación:

- (I). Un programa que filtre paquetes HTTP con código de respuesta mayor a 400.
- (II). Un programa que calcule estadísticas de tráfico por IP origen.
- (III). Un programa que detecte intentos de escaneo de puertos.
- (IV). Un programa que identifique las 10 conexiones más lentas en una captura.
- (V). Un programa que genere un resumen de protocolos utilizados por tiempo.
- (VI). Un programa que detecte transferencias de archivos grandes.
- (VII). Un programa que identifique hosts que generan tráfico anómalo.
- (VIII). Un programa que analice latencia entre solicitudes y respuestas HTTP.
- (IX). Un programa que extraiga y analice cookies HTTP específicas.
- (X). Un programa que detecte patrones de comunicación periódicos.

Casos de rechazo:

- (I). Un programa malformado sintácticamente.
- (II). Un programa que intente acceder a campos de protocolos inexistentes.
- (III). Un programa que intente capturar desde una interfaz que no existe.
- (IV). Un programa que mezcle tipos de datos incompatibles.
- (V). Un programa que intente realizar operaciones no permitidas sobre ciertos tipos.

6. Ejemplos

```
// Capturar tráfico HTTP en tiempo real
capture from "eth0" where dst_port == 80 or dst_port == 443;

// Extraer información relevante
extract {
    timestamp,
    ip.src_addr as client_ip,
    http.method,
    http.uri,
    http.response_code as status
};

// Filtrar solo respuestas con error
filter status >= 400;

// Agrupar por URI y contar errores
group by http.uri;
count errors;

// Generar alerta si hay muchos errores 500
alert when count(status >= 500) > 5 in 1min
    message "Múltiples errores de servidor en ${http.uri}";

// Exportar resultados
export to "http_errors.json";
```

```
// Importar archivo PCAP existente
import from "capture.pcap";

// Definir patrón de escaneo
define scan_pattern {
    same ip.src_addr,
    different tcp.dst_port,
    count(tcp.dst_port) > 20,
    timespan < 5s
};

// Extraer información relevante
extract {
    ip.src_addr as scanner_ip,
    count(distinct tcp.dst_port) as ports_scanned,
    min(timestamp) as start_time,
    max(timestamp) as end_time
};

// Agrupar por IP origen
group by scanner_ip;

// Filtrar por patrón de escaneo
filter matches scan_pattern;

// Generar reporte
report {
    table scanner_ip, ports_scanned, start_time ordered by ports_scanned desc,
    timeline scanner_ip by timestamp
};

// Exportar IPs sospechosas
export scanner_ip to "suspicious_ips.txt";
```